QUIC versus TCP+TLS under Client Mobility: A Review of Handshake Latency, Congestion Control and Connection Migration, with a Proposal for Migration-Aware Careful Resume

Shashank Tiwari
Reg. No. 25BDS0053
School of Computer Science and Engineering
VIT, Vellore, India

Tanmay Dawande
Reg. No. 25BCE2125
School of Computer Science and Engineering
VIT, Vellore, India

Riddhi Gupta
Reg. No. 25BCE2157
School of Computer Science and Engineering
VIT, Vellore, India

Kyra Panwar
Reg. No. 25BBS0095
School of Computer Science and Engineering
VIT, Vellore, India

Team MACR | BACSE203 Computer Networks, Fall Semester 2026 | Faculty: Dr. T. Thiruvenkadam

Case Study Assignment: Final Report (Part A: Review Paper; Part B: Simulation-Based Case Study; Part C: Alignment and Limitations Note)

INDIVIDUAL CONTRIBUTION STATEMENT

No.	Member	Reg. No.	Individual contribution
1	Shashank Tiwari	25BDS0053	Abstract; Section I (Introduction); Section V (Alignment and Limitations Note, Part C); Section VI (Conclusion and Future Scope); overall integration, consistency checking and final formatting.
2	Tanmay Dawande	25BCE2125	Section II (Literature Review, Part A) and the synthesis of agreements, disagreements and gaps in Section III-A (Table I); selection of sources, reference list, consistent IEEE citation style and the similarity-index check.
3	Riddhi Gupta	25BCE2157	Sections III-B and III-C (QUIC vs TCP+TLS comparison, Table II, and the MACR proposal); Part B set-up: Wireshark, TLS key-log and Chrome configuration (Section IV-A) and the design of the handshake experiment (Section IV-B).
4	Kyra Panwar	25BBS0095	Section III-D (evidence from the MACR model: Table III, Figs. 1–2); Part B execution: running the handshake experiment and the MACR model (Sections IV-B, IV-C), capturing and captioning all screenshots, and filling Tables IV–V.

***Abstract*—Web traffic is moving from HTTP/2 over TCP and TLS to HTTP/3 over QUIC, a UDP-based transport that folds the TLS 1.3 handshake into its own and lets a connection survive a change of client address. This review compares the two stacks across four themes drawn from 26 sources, 15 of them RFCs or peer-reviewed papers: connection set-up, stream multiplexing, congestion control and fairness, and behaviour when a mobile client changes network. The literature agrees that QUIC saves one round trip at connection set-up and removes transport-level head-of-line blocking, but disagrees on how much of QUIC's measured advantage comes from the protocol itself rather than from the congestion controller and implementation that accompany it. On mobility the studies converge on a gap: QUIC keeps the connection alive across migration but resets its congestion controller, while the newly standardised Careful Resume method reuses saved congestion state only for a later connection. We argue that this gap matters most for clients that move back and forth between two networks, and we propose Migration-Aware Careful Resume (MACR), which caches congestion state per path and re-applies it through Careful Resume's safety phases. In a packet-level model covering 2,580 return migrations, MACR removed slow-start overshoot losses on uncontended paths (340 to 0 packets per return) and cut ramp-up time from 1.0 s to 0.6 s on a 100 Mb/s, 100 ms path, but was 48% slower than a reset against an established loss-based competitor.**

***Index Terms*—QUIC, TCP, TLS 1.3, HTTP/3, connection migration, congestion control, Careful Resume, mobility.**

I. INTRODUCTION

For three decades the web has used TCP for reliable delivery, with TLS layered on top for security. QUIC replaces this pair with a single transport that runs over UDP, encrypts almost all of its own headers and carries several independent streams in one connection [1, 2]. Google reported that QUIC already carried an estimated 7% of Internet traffic in 2017 [1], and the IETF standardised it in 2021, with HTTP/3 defined on top of it a year later [3].

The design choices behind QUIC target three weaknesses of TCP+TLS: the separate transport and security handshakes, head-of-line blocking when several HTTP streams share one TCP byte stream, and the binding of a connection to a fixed four-tuple of addresses and ports. The third weakness matters more every year, because phones and laptops routinely switch between Wi-Fi and cellular access. A TCP connection breaks when the client's address changes; a QUIC connection can migrate to the new path [2].

This report asks three questions. Where does the literature agree that QUIC outperforms TCP+TLS, and why? Where do measurement studies disagree? And what remains unsolved when a mobile client changes path? We searched IEEE Xplore, the ACM Digital Library, the RFC Editor and arXiv for work published mainly between 2017 and 2026, and kept 26 sources: 15 RFCs and IETF documents, ten peer-reviewed papers, and one implementation issue report. Section II reviews the literature by theme; Section III compares the findings, identifies agreements, disagreements and gaps, and presents our own inference together with a proposal and its evaluation; Section IV reports a simulation-based case study with Wireshark and our packet-level model (Part B); Section V relates its results to the literature and states its limitations (Part C); Section VI concludes.

II. Literature Review (Part A)

A. Connection set-up and security

With TCP and TLS 1.3, a client must complete TCP's three-way handshake before it can send the TLS ClientHello, so the first request leaves after two round trips [4]. QUIC carries the TLS 1.3 handshake inside its own packets, so transport and cryptographic set-up complete together in one round trip, and a client that has spoken to the server before can send data in its first flight (0-RTT) [5, 2]. Both stacks therefore offer the same TLS 1.3 security properties, but QUIC additionally encrypts most transport headers, which hides packet numbers and acknowledgements from middleboxes [2].

The size of the benefit depends on the transfer. Yu and Benson tested production QUIC endpoints from Google, Facebook and Cloudflare and found that the one-round-trip saving gave HTTP/3 a clear lead for small payloads but made little difference above roughly 1 MB on a 10 Mb/s link [6]. The handshake advantage is real but bounded by the share of a transfer's lifetime spent setting up.

B. Stream multiplexing and head-of-line blocking

HTTP/2 multiplexes many requests over one TCP connection, so a single lost segment stalls every stream until it is retransmitted, because TCP must deliver bytes in order. QUIC tracks each stream separately, so a loss only delays the stream it belongs to [2, 3]. Early measurements showed QUIC improving page-load times across many network conditions on desktop clients [7], and Google attributed part of its gains on lossy networks to the removal of this blocking [1].

C. Congestion control, implementation and fairness

QUIC does not define a new congestion-control algorithm. RFC 9002 specifies a NewReno-style controller, and deployments commonly use CUBIC [8] or BBR [9] instead [10]. Because QUIC runs in user space, each implementation ships its own congestion controller, loss detection and pacing, and the literature shows that these choices drive much of the measured behaviour. Kakhki *et al.* found that QUIC with CUBIC consistently took more than half of a bottleneck shared with TCP CUBIC flows, because its window grew more aggressively; that its advantage shrank on phones, where the user-space process spent 58% of its time application-limited; and that it misread network reordering as loss [7]. Yu and Benson traced several HTTP/2-versus-HTTP/3 differences to congestion-control choices and implementation bugs rather than to protocol design [6]. Studies of fast-changing satellite paths add that even modern model-based controllers such as BBRv3 adapt slowly to abrupt RTT changes [11].

D. Mobility and connection migration

Because a TCP connection is identified by its address four-tuple, a change of client address ends it; Multipath TCP adds subflows over several addresses but needs support at both ends and in middleboxes [12]. QUIC instead identifies a connection by connection IDs. After an address change the server validates the new path with PATH_CHALLENGE and PATH_RESPONSE frames, and the connection continues [2]. Deployment lags the design: Internet-wide scans in 2025 found that several of the most popular QUIC servers did not yet support migration [13]. Research has proposed migrating proactively to the best available path [14] and adapting the congestion window when the client's address changes [15], and the IETF is completing a multipath extension that keeps a congestion controller per active path [16]. On satellite links, where handovers shift the RTT without changing the address, SaTCP and StarQUIC freeze the congestion window around predicted handovers [17, 18] and LeoCC re-estimates its path model after each reconfiguration [19].

A detail of the standard links mobility to congestion control. RFC 9000 requires an endpoint to reset its congestion controller and RTT estimator after validating a new peer address, exempting only port-only changes caused by NAT rebinding, and warns that state reused on a different path can make a sender too aggressive [2]. Implementations do not always follow this: an open picoquic issue reports that, after a code refactor, migrating connections kept the old path's window [20].

E. Reusing congestion state

Slow start begins from an initial window of about ten packets [21] and doubles each round trip, overshooting the bottleneck buffer when it reaches capacity unless a heuristic such as HyStart++ exits early [22]. Several standards let a sender skip part of this cost by reusing state: TCP control-block sharing reuses parameters across connections to the same host [23], RFC 7661 governs restart after idle periods [24], and Careful Resume, published as RFC 9959 in 2026, defines a safe procedure for jumping to half of a previously measured window, paced over one RTT, with reconnaissance, validation and safe-retreat phases [25]. Its first detailed evaluation reports large start-up gains on satellite paths and good interaction with HyStart++ [26]. Careful Resume, however, is scoped to a *subsequent connection* and must stop if the path changes.

III. Comparative Discussion and Analysis (Part A)

A. Agreements, disagreements and gaps

Table I summarises how the reviewed studies relate to one another.

TABLE I
Synthesis of the reviewed literature

Theme	Where the literature agrees	Where it disagrees or is mixed	Gap identified
Connection set-up	QUIC saves one RTT over TCP+TLS 1.3 and supports 0-RTT [4, 5]	Size of the end-user benefit: large for small objects, negligible above ~1 MB [6]	Few studies on handshake cost during frequent re-connections
Multiplexing	Per-stream delivery removes transport HOL blocking [2, 3]	—	—
Congestion control	QUIC reuses TCP algorithms [10]	Whether QUIC wins because of protocol design [1] or because of CC and implementation choices [7, 6]; fairness to TCP [7]	Evaluations rarely separate protocol from CC

Theme	Where the literature agrees	Where it disagrees or is mixed	Gap identified
Mobility	Migration keeps the connection alive [2]; TCP needs MPTCP [12]	Freeze vs reset vs re-estimate on path change [17, 19]	Deployment incomplete [13]; CC state reset on every migration
State reuse	Saved state can shorten start-up safely [23, 25, 26]	—	No standard procedure for reuse *within* a connection after migration

Three findings stand out. First, the studies agree on QUIC's structural advantages (one-RTT set-up, per-stream delivery, migration). Second, they disagree on attribution: the Google deployment report credits the protocol [1], whereas independent measurements show that congestion-control choice and implementation quality explain much of the difference, and can make QUIC unfair to TCP [7, 6]. Our reading is that "QUIC versus TCP" comparisons are only meaningful when the congestion controller is held fixed. Third, the mobility literature exposes a gap between two standards: RFC 9000 resets congestion state on migration, and RFC 9959 reuses state only across connections. Neither covers a client that returns to a path it used seconds earlier.

B. Side-by-side comparison

TABLE II
QUIC VERSUS TCP+TLS 1.3 ON THE REVIEWED DIMENSIONS

Dimension	TCP + TLS 1.3	QUIC (HTTP/3)
Round trips before first request	2 (TCP SYN, then TLS) [4]	1; 0 on resumption [5]
Header encryption	TCP header in clear	Most transport fields encrypted [2]
Loss affects	All multiplexed streams	Only the affected stream [3]
Congestion control	In kernel (CUBIC, BBR)	In user space, per implementation [10, 7]
Address change	Connection breaks (unless MPTCP) [12]	Connection migrates; CC reset [2]
Reuse of path knowledge	Across connections [23]	Across connections (Careful Resume) [25]; not on migration

C. Our inference and proposal: Migration-Aware Careful Resume

The reset rule in RFC 9000 is sound for a path the sender has never observed. It is wasteful when a client oscillates between, for example, home Wi-Fi and cellular data, because each return to a path re-runs slow start from ten packets even though that path was measured seconds earlier. On a high bandwidth-delay-product path the cost is twofold: slow start needs many RTTs to reach capacity, and its final doubling overflows the buffer. We therefore propose MACR, which combines the two standards rather than inventing a new mechanism.

- **Per-path cache.** When the connection leaves path p, the sender stores the capacity it actually delivered, W_p = min(*cwnd*, $N_{del}(1\text{ s}) \cdot R_p$ / 1 s) packets, and the minimum RTT R_p, where $N_{del}(1\text{ s})$ is the number of packets delivered in the last second (1). Entries expire after a lifetime L.
- **Careful Resume re-entry.** On a return to p, the sender sends one initial window (reconnaissance) and checks that the RTT is within ($R_p/2$, $10R_p$). It then jumps to $J = \min(J_{max}, W_p/2)$ packets, paced at one packet every $\Delta_{pkt} = srtt/J$ seconds (2), and validates the jump before continuing; congestion during validation triggers a safe retreat to half the validated pipe, as in RFC 9959 [25].
- **Overshoot guard and quarantine.** MACR sets *ssthresh* = max(W_p, J) so growth beyond the cached capacity is linear, and disables reuse on a path for Q seconds after a retreat.

D. Evidence from a packet-level model

We tested MACR in a discrete-event simulator of two access paths (50 Mb/s, 40 ms and 20 Mb/s, 60 ms, drop-tail buffers of one BDP) and a client that switches between them every few seconds. Senders follow RFC 9002 NewReno with pacing; four policies are compared: RFC 9000 reset, carrying the previous path's window, restoring the cached window without validation, and MACR. Seven scenarios and a buffer sweep give 860 runs and 2,580 return migrations; significance uses Wilcoxon signed-rank tests paired by seed (n = 20). Section IV-C reproduces these runs on a team computer. Table III and Fig. 1–2 give the main results.

TABLE III
MEDIAN OUTCOME PER RETURN MIGRATION (FIRST 2 S), MACR VS RFC 9000 RESET

Scenario	Policy	T90 (s)	Data in 2 s (MB)	Own losses	Peak queue (ms)
Uncontended, 50 Mb/s	Reset	0.40	11.07	340	41.5
	MACR	0.40	11.52	0	8.8
High BDP, 100 Mb/s, 100 ms	Reset	1.00	15.69	1,674	104.2
	MACR	0.60	19.42	0	0.4
Two flows joined while away	Reset	3.70	3.03	21	41.5
	MACR	3.15	2.95	8	41.5
Established competitor	Reset	2.25	4.62	48	41.5
	MACR	5.00	2.39	1	40.9

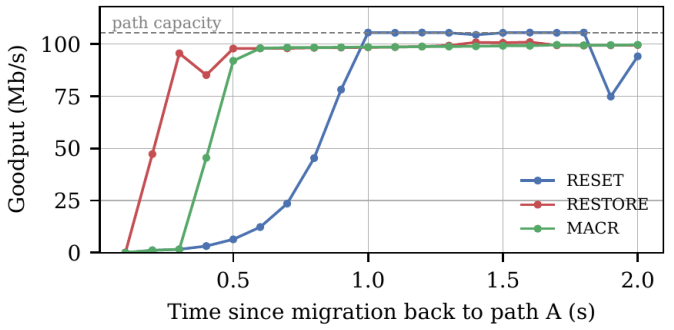


Fig. 1. Goodput after one return to the 100 Mb/s, 100 ms path. MACR spends one RTT in reconnaissance and then reaches capacity without the slow-start overshoot that follows a reset.

Three results are relevant to the review. (i) On paths that did not change, MACR removed slow-start overshoot losses (340 and 1,674 packets per return to 0) and cut peak queuing delay by 79–99%; on the high-BDP path it also shortened the time to reach 90% throughput by 40% ($p < 10^{-4}$) and delivered 23.8% more data in 2 s. These gains held for buffers from 0.25 to 2

BDP, whereas restoring the window *without pacing* stalled below 90% throughput on shallow buffers. (ii) When two new flows had joined the path, MACR's validation triggered a safe retreat on 35% of returns; median own losses fell from 21 to 8, but the difference was not significant ($p = 0.06$), so we read it as "no harm", not "proven help". (iii) Against an established loss-based flow MACR was 48% slower to claim its share, because a reset's overshoot forces the competitor to back off while MACR's paced entry does not. This mirrors the fairness tension in the congestion-control literature [7]: a gentler sender is also a slower one.

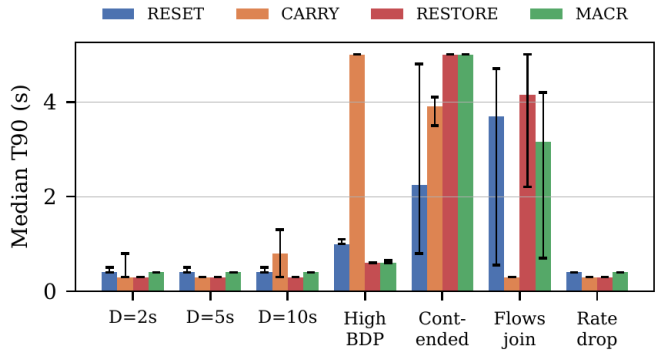


Fig. 2. Median time to reach 90% throughput (T90) per scenario and policy, with 95% bootstrap confidence intervals (60 returns each; censored at the 5 s dwell time).

IV. Simulation-Based Case Study (Part B)

Tool selection. Mininet requires a Linux host, which was not available to the team, and Cisco Packet Tracer cannot generate QUIC or HTTP/3 traffic, so this study uses **Wireshark 4.6.9** to capture real HTTPS connections from Google Chrome. MACR itself requires changes inside a QUIC stack that no browser exposes, so it cannot be demonstrated in Wireshark; it is evaluated instead with the team's packet-level model (Section IV-C). This limitation is discussed in Section V.

Experiment plan. Section IV-B compares connection set-up over TCP+TLS 1.3 and over QUIC to the same server (Section II-A). Section IV-C runs the MACR model and reproduces the results of Section III-D.

A. Tools and set-up

Software: Wireshark 4.6.9 (with the ChmodBPF capture helper), Google Chrome and a command-line terminal, on a laptop connected to the campus Wi-Fi (interface en0). Chrome was started from Terminal with a TLS key-log file and a fresh temporary profile for every run, so that no cached session could be reused and Wireshark could decrypt the TLS handshake:

```
# TCP+TLS run (QUIC disabled)
export SSLKEYLOGFILE=$HOME/cs_keys.log
"/Applications/Google Chrome.app/Contents/MacOS/Google
Chrome" --user-data-dir=/tmp/cs_$RANDOM \
    --disable-quic https://cloudflare-quic.com


# QUIC run (QUIC forced for this origin)
"/Applications/Google Chrome.app/Contents/MacOS/Google
Chrome" --user-data-dir=/tmp/cs_$RANDOM \
    --enable-quic --origin-to-force-quic-on=cloudflare-
quic.com:443 https://cloudflare-quic.com
```

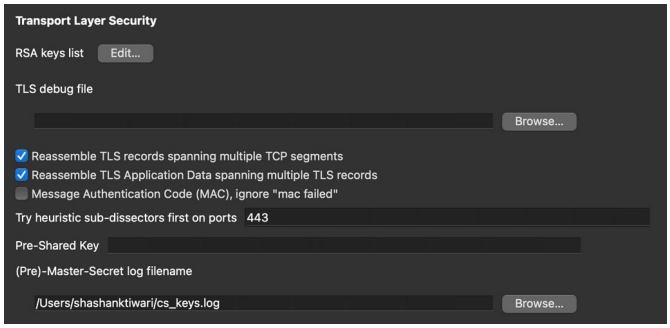


Fig. 3. Wireshark TLS preferences: the (Pre)-Master-Secret log file written by Chrome is loaded so that the TLS 1.3 and QUIC handshakes can be dissected.

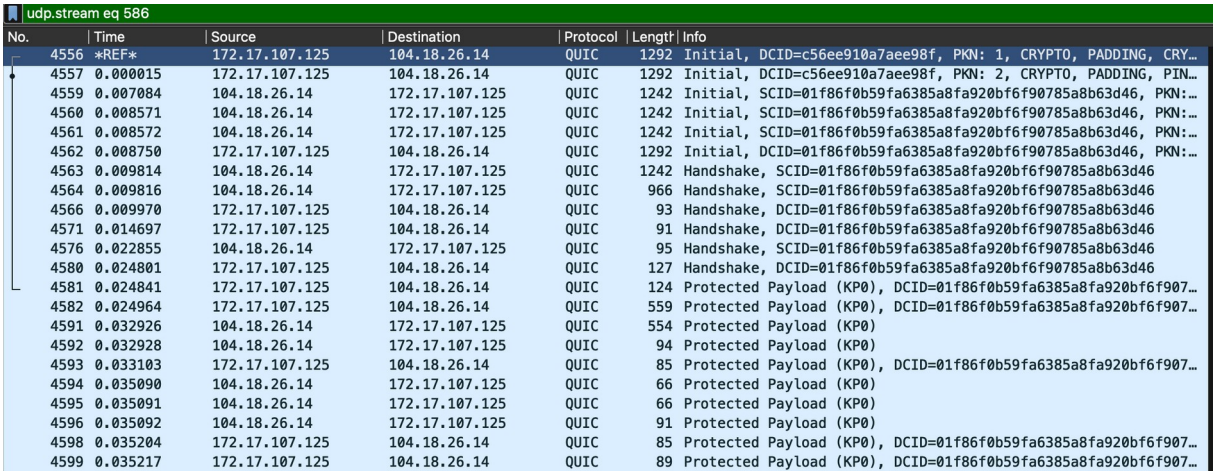
B. Experiment: connection set-up, TCP+TLS 1.3 vs QUIC

Steps. (1) Start a capture on Wi-Fi: en0. (2) Launch Chrome with *--disable-quic* and load https://cloudflare-quic.com; stop the capture. (3) Isolate the connection with the display filter *tls.handshake.extensions_server_name contains "cloudflare"* followed by Conversation Filter → TCP, and set a time reference (Cmd+T) on the first packet. (4) Repeat with QUIC forced on, isolating the connection with Conversation Filter → UDP. (5) Read, for each connection, when the client finished its handshake, when the request left and when the first response arrived.

tcp.stream eq 26

No.	Time	Source	Destination	Protocol	Length	Info
6287	*REF*	172.17.107.125	104.18.27.14	TCP	78	52490 → 443 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 W…
6294	0.007814	104.18.27.14	172.17.107.125	TCP	74	443 → 52490 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 MSS=…
6295	0.007893	172.17.107.125	104.18.27.14	TCP	66	52490 → 443 [ACK] Seq=1 Ack=1 Win=131904 Len=0 TSval=1085153…
6296	0.008150	172.17.107.125	104.18.27.14	TCP	1454	52490 → 443 [ACK] Seq=1 Ack=1 Win=131904 Len=1388 TSval=1085…
6297	0.008155	172.17.107.125	104.18.27.14	TLSv1…	637	Client Hello (SNI=cloudflare-quic.com)
6302	0.020139	104.18.27.14	172.17.107.125	TCP	66	443 → 52490 [ACK] Seq=1 Ack=1960 Win=139264 Len=0 TSval=2520…
6307	0.022064	104.18.27.14	172.17.107.125	TLSv1…	1514	Server Hello, Change Cipher Spec
6308	0.022065	104.18.27.14	172.17.107.125	TCP	1514	443 → 52490 [ACK] Seq=1449 Ack=1960 Win=139264 Len=1448 TSva…
6309	0.022066	104.18.27.14	172.17.107.125	TLSv1…	428	Encrypted Extensions, Compressed Certificate, Certificate Ve…
6310	0.022135	172.17.107.125	104.18.27.14	TCP	66	52490 → 443 [ACK] Seq=1960 Ack=3259 Win=128704 Len=0 TSval=1…
6321	0.034423	172.17.107.125	104.18.27.14	TLSv1…	130	Change Cipher Spec, Finished
6328	0.034875	172.17.107.125	104.18.27.14	HTTP2	158	Magic, SETTINGS[0], WINDOW_UPDATE[0]
6329	0.034898	172.17.107.125	104.18.27.14	HTTP2	575	HEADERS[1]: GET /
6337	0.043020	104.18.27.14	172.17.107.125	TCP	66	443 → 52490 [ACK] Seq=3259 Ack=2116 Win=139264 Len=0 TSval=2…
6338	0.043021	104.18.27.14	172.17.107.125	HTTP2	603	SETTINGS[0], WINDOW_UPDATE[0], SETTINGS[0]
6339	0.043092	172.17.107.125	104.18.27.14	TCP	66	52490 → 443 [ACK] Seq=2625 Ack=3796 Win=130560 Len=0 TSval=1…
6340	0.043487	172.17.107.125	104.18.27.14	HTTP2	97	SETTINGS[0]
6355	0.053902	104.18.27.14	172.17.107.125	HTTP2	195	HEADERS[1]: 200 OK
6356	0.054092	172.17.107.125	104.18.27.14	TCP	66	52490 → 443 [ACK] Seq=2656 Ack=3925 Win=130944 Len=0 TSval=1…
6357	0.055514	104.18.27.14	172.17.107.125	TLSv1…	1457	[TLS segment of a reassembled PDU]
6358	0.055516	104.18.27.14	172.17.107.125	TLSv1…	1514	[TLS segment of a reassembled PDU]
6359	0.055517	104.18.27.14	172.17.107.125	TLSv1…	1400	[TLS segment of a reassembled PDU]

Fig. 4. TCP+TLS 1.3 connection to cloudflare-quic.com (104.18.27.14). The TCP handshake (SYN, SYN-ACK, ACK) must complete before the TLS Client Hello is sent; the client's Finished leaves at 34.4 ms, the HTTP/2 GET at 34.9 ms and the 200 OK arrives at 53.9 ms.



udp.stream eq 586

No.	Time	Source	Destination	Protocol	Lengtl	Info
4556	*REF*	172.17.107.125	104.18.26.14	QUIC	1292	Initial, DCID=c56ee910a7aee98f, PKN: 1, CRYPTO, PADDING, CRY…
4557	0.000015	172.17.107.125	104.18.26.14	QUIC	1292	Initial, DCID=c56ee910a7aee98f, PKN: 2, CRYPTO, PADDING, PIN…
4559	0.007084	104.18.26.14	172.17.107.125	QUIC	1242	Initial, SCID=01f86f0b59fa6385a8fa920bf6f90785a8b63d46, PKN:…
4560	0.008571	104.18.26.14	172.17.107.125	QUIC	1242	Initial, SCID=01f86f0b59fa6385a8fa920bf6f90785a8b63d46, PKN:…
4561	0.008572	104.18.26.14	172.17.107.125	QUIC	1242	Initial, SCID=01f86f0b59fa6385a8fa920bf6f90785a8b63d46, PKN:…
4562	0.008750	172.17.107.125	104.18.26.14	QUIC	1292	Initial, DCID=01f86f0b59fa6385a8fa920bf6f90785a8b63d46, PKN:…
4563	0.009814	104.18.26.14	172.17.107.125	QUIC	1242	Handshake, SCID=01f86f0b59fa6385a8fa920bf6f90785a8b63d46
4564	0.009816	104.18.26.14	172.17.107.125	QUIC	966	Handshake, SCID=01f86f0b59fa6385a8fa920bf6f90785a8b63d46
4566	0.009970	172.17.107.125	104.18.26.14	QUIC	93	Handshake, DCID=01f86f0b59fa6385a8fa920bf6f90785a8b63d46
4571	0.014697	172.17.107.125	104.18.26.14	QUIC	91	Handshake, DCID=01f86f0b59fa6385a8fa920bf6f90785a8b63d46
4576	0.022855	104.18.26.14	172.17.107.125	QUIC	95	Handshake, SCID=01f86f0b59fa6385a8fa920bf6f90785a8b63d46
4580	0.024801	172.17.107.125	104.18.26.14	QUIC	127	Handshake, DCID=01f86f0b59fa6385a8fa920bf6f90785a8b63d46
4581	0.024841	172.17.107.125	104.18.26.14	QUIC	124	Protected Payload (KP0), DCID=01f86f0b59fa6385a8fa920bf6f907…
4582	0.024964	172.17.107.125	104.18.26.14	QUIC	559	Protected Payload (KP0), DCID=01f86f0b59fa6385a8fa920bf6f907…
4591	0.032926	104.18.26.14	172.17.107.125	QUIC	554	Protected Payload (KP0)
4592	0.032928	104.18.26.14	172.17.107.125	QUIC	94	Protected Payload (KP0)
4593	0.033103	172.17.107.125	104.18.26.14	QUIC	85	Protected Payload (KP0), DCID=01f86f0b59fa6385a8fa920bf6f907…
4594	0.035090	104.18.26.14	172.17.107.125	QUIC	66	Protected Payload (KP0)
4595	0.035091	104.18.26.14	172.17.107.125	QUIC	66	Protected Payload (KP0)
4596	0.035092	104.18.26.14	172.17.107.125	QUIC	91	Protected Payload (KP0)
4598	0.035204	172.17.107.125	104.18.26.14	QUIC	85	Protected Payload (KP0), DCID=01f86f0b59fa6385a8fa920bf6f907…
4599	0.035217	172.17.107.125	104.18.26.14	QUIC	89	Protected Payload (KP0), DCID=01f86f0b59fa6385a8fa920bf6f907…

Fig. 5. QUIC connection to the same site (104.18.26.14). The client's first packet is a QUIC Initial carrying the TLS Client Hello; the server answers with Initial and Handshake packets at 7.1–9.8 ms and the client completes the handshake at 10.0 ms, one round trip after it started.

TABLE IV
HANDSHAKE EXPERIMENT RESULTS (ONE RUN PER PROTOCOL, TIMES FROM THE FIRST CLIENT PACKET)

Measurement	TCP + TLS 1.3	QUIC
Round-trip time (first reply from server)	7.8 ms (SYN-ACK)	7.1 ms (server Initial)
Client finishes handshake	34.4 ms (TLS Finished)	10.0 ms (QUIC Handshake)
Request sent	34.9 ms (HTTP/2 GET)	≈ 25.0 ms*
First response data	53.9 ms (HTTP/2 200 OK)	≈ 32.9 ms*
Round trips before the request can be sent	2	1

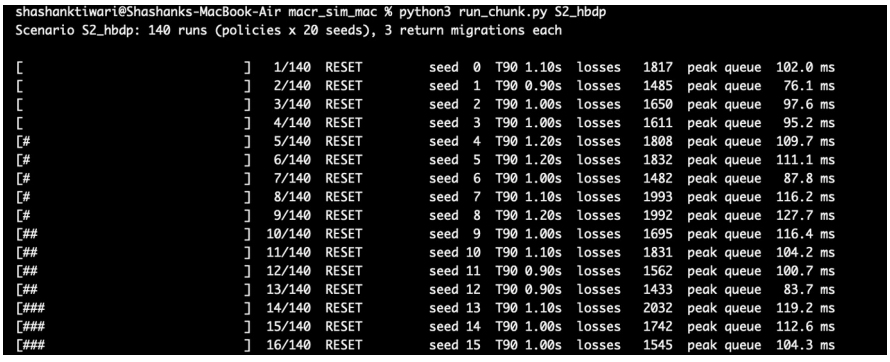
*The QUIC application packets were shown as "Protected Payload" because Wireshark did not decrypt the 1-RTT keys for this connection, so the request and response were identified by direction and size (559-byte client packet, 554-byte server packet).

Result. The capture reproduces the handshake structure in the standards. With TCP+TLS the client needed one round trip for TCP and a second for TLS before it could send the request; with QUIC the TLS 1.3 handshake travelled inside the first QUIC packets and completed after one round trip. In this run QUIC's first response arrived about 21 ms (≈ 39%) earlier. The saving is larger than one 7–8 ms round trip, so part of it comes from client and server processing time, which a single run cannot separate (see Section V).

C. MACR model run

MACR cannot be tested inside Chrome, so the packet-level model of Section III-D was run on a team member's computer (Python 3 with numpy, scipy and matplotlib). The model is deterministic for fixed seeds, so the output matches the numbers in Part A exactly.

```
cd ~/Desktop/macr_sim_mac
python3 -m venv .venv && source .venv/bin/activate && pip
install numpy scipy matplotlib
python3 run_chunk.py S2_hbdp      # one scenario with live
progress (also S1_d2 ... S5_rate)
python3 run_chunk.py merge && python3 sweep.py && python3
analyse.py
python3 summary_tables.py         # prints the result table
and Wilcoxon tests
```



```
shashanktiwari@Shashanks-MacBook-Air macr_sim_mac % python3 run_chunk.py S2_hbdp
Scenario S2_hbdp: 140 runs (policies x 20 seeds), 3 return migrations each

[                    ]   1/140  RESET        seed  0  T90 1.10s  losses   1817  peak queue  102.0 ms
[                    ]   2/140  RESET        seed  1  T90 0.90s  losses   1485  peak queue   76.1 ms
[                    ]   3/140  RESET        seed  2  T90 1.00s  losses   1650  peak queue   97.6 ms
[                    ]   4/140  RESET        seed  3  T90 1.00s  losses   1611  peak queue   95.2 ms
[#                   ]   5/140  RESET        seed  4  T90 1.20s  losses   1808  peak queue  109.7 ms
[#                   ]   6/140  RESET        seed  5  T90 1.20s  losses   1832  peak queue  111.1 ms
[#                   ]   7/140  RESET        seed  6  T90 1.00s  losses   1482  peak queue   87.8 ms
[#                   ]   8/140  RESET        seed  7  T90 1.10s  losses   1993  peak queue  116.2 ms
[#                   ]   9/140  RESET        seed  8  T90 1.20s  losses   1992  peak queue  127.7 ms
[##                  ]  10/140  RESET        seed  9  T90 1.00s  losses   1695  peak queue  116.4 ms
[##                  ]  11/140  RESET        seed 10  T90 1.10s  losses   1831  peak queue  104.2 ms
[##                  ]  12/140  RESET        seed 11  T90 0.90s  losses   1562  peak queue  100.7 ms
[##                  ]  13/140  RESET        seed 12  T90 0.90s  losses   1433  peak queue   83.7 ms
[###                 ]  14/140  RESET        seed 13  T90 1.10s  losses   2032  peak queue  119.2 ms
[###                 ]  15/140  RESET        seed 14  T90 1.00s  losses   1742  peak queue  112.6 ms
[###                 ]  16/140  RESET        seed 15  T90 1.00s  losses   1545  peak queue  104.3 ms
```

Fig. 6. The model running the high-BDP scenario (100 Mb/s, 100 ms) on a team member's computer: each line is one run of three return migrations. Under the RFC 9000 reset, each run loses about 1,400–2,000 packets.

```
TABLE B4 - median outcomes per return migration (first 2 s)
Scenario  Policy     T90(s)     MB2  OwnLoss  CompLoss  PeakQ(ms)
S1_d5     RESET        0.40   11.07      340       nan       41.5
S1_d5     CARRY        0.30   11.85        0       nan       12.7
S1_d5     RESTORE      0.30   11.79        0       nan       38.1
S1_d5     MACR         0.40   11.52        0       nan        8.8
S2_hbdp   RESET        1.00   15.69     1674       nan      104.2
S2_hbdp   CARRY        5.00    4.68        0       nan       -0.0
S2_hbdp   RESTORE      0.60   21.28        0       nan       92.9
S2_hbdp   MACR         0.60   19.42        0       nan        0.4
S3_cont   RESET        2.25    4.62       48      47.0       41.5
S3_cont   CARRY        3.90    4.90       38      37.5       41.5
S3_cont   RESTORE      5.00    2.64        1       2.0       41.1
S3_cont   MACR         5.00    2.39        1       2.0       40.9
S4_join   RESET        3.70    3.03       21      38.5       41.5
S4_join   CARRY        0.30    4.21       57      68.0       41.5
S4_join   RESTORE      4.15    2.55       16       3.5       41.2
S4_join   MACR         3.15    2.95        8      27.5       41.5
S5_rate   RESET        0.40    4.65      246       nan      104.4
S5_rate   CARRY        0.30    4.85        0       nan       70.9
S5_rate   RESTORE      0.30    4.83        0       nan       40.5
S5_rate   MACR         0.40    4.68        0       nan       20.9

Seed-level Wilcoxon signed-rank tests, MACR vs RESET (n=20 seeds)
  S1_d5    qpeak_ms   p = 1.9e-06
  S2_hbdp  t90        p = 7.9e-05
  S2_hbdp  mb2        p = 1.9e-06
  S3_cont  mb2        p = 1.9e-06
  S4_join  main_loss  p = 0.062
  S4_join  comp_loss  p = 0.49
(.venv) shashanktiwari@Shashanks-MacBook-Air macr_sim_mac %
```

Fig. 7. Summary printed by the model: median outcome per return migration for each scenario and policy, and seed-level Wilcoxon tests of MACR against reset. The values match Table III.

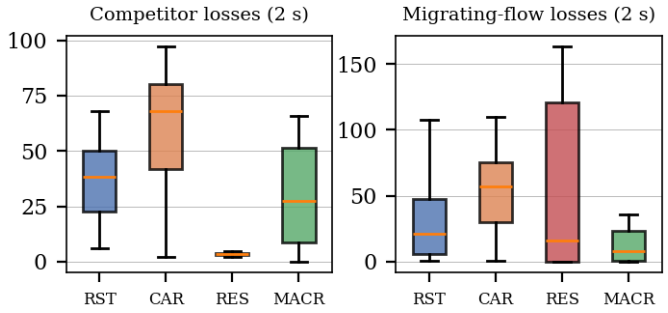


Fig. 8. Losses in the first 2 s after returning to a path that two new flows had joined. Carrying the old window (CAR) raised competitor losses; MACR kept its own losses lowest without raising the competitors'.

TABLE V
CONDITION A (RFC 9000 RESET) VERSUS CONDITION B (MACR), HIGH-BDP PATH

Metric (median per return)	A: Reset	B: MACR	Change
Time to 90% throughput (s)	1.00	0.60	−40%
Data delivered in first 2 s (MB)	15.69	19.42	+23.8%
Own packet losses	1,674	0	−100%
Peak queuing delay (ms)	104.2	0.4	−99.6%

V. ALIGNMENT AND LIMITATIONS NOTE (PART C)

A. How do the Part B results compare with the literature?

Connection set-up (Section IV-B). The captures are **consistent** with the standards and the measurement literature. Fig. 4 shows TCP+TLS 1.3 completing a TCP round trip before the TLS Client Hello could be sent, and Fig. 5 shows QUIC carrying the TLS 1.3 handshake in its first flight and finishing after one round trip [4, 5]. The first response arrived about 21 ms earlier over QUIC (32.9 ms vs 53.9 ms). Loading a single page from Vellore to Cloudflare's nearby edge is a small, set-up-dominated transfer, which is exactly where Yu and Benson found the handshake saving matters most [6]. One observation also matches the deployment literature: the browser's own background traffic to Google already used QUIC with 0-RTT resumption, showing how widely QUIC is deployed [1].

MACR model (Section IV-C). The model is **partially consistent** with the literature. It agrees with the Careful Resume evaluation that a paced, validated jump reaches capacity faster than slow start without overshoot [26], and with RFC 9000's warning that state reused on a changed path can be too aggressive [2]: the unvalidated "carry" policy raised competitor losses by 77%, while MACR's validation retreated on 35% of returns. It also reproduces a tension from the measurement literature: a sender that avoids overshoot converges more slowly against a loss-based competitor, the mirror image of QUIC's aggressiveness toward TCP reported by Kakhki *et al.* [7].

B. What could the simulation not capture, and why?

- **MACR on a real stack.** No browser lets us change QUIC's congestion state on migration, so MACR was evaluated only in a model, not on live traffic.
- **Real migrations.** Wireshark on one laptop cannot force Chrome to migrate a QUIC connection between Wi-Fi and a phone hotspot in a controlled, repeatable way, so connection migration itself was not captured.
- **Internet variability and a single run.** The handshake experiment ran once per protocol over the public Internet, and the two connections reached different Cloudflare addresses (104.18.27.14 and 104.18.26.14). Cross traffic, server load and Wi-Fi conditions add noise we could not control, so the 21 ms difference is an illustration, not a measured average.
- **Partial decryption.** Wireshark decrypted the TLS-over-TCP traffic but showed QUIC application data only as "Protected Payload", so the QUIC request and response times were identified by packet size and direction.
- **Scale.** All experiments involve one client and at most three competing flows, far from the thousands of flows that share real bottlenecks.

C. Simplified substitutes and what they leave out

The main substitution is the Python model of Section IV-C, which stands in for a modified QUIC implementation. It uses NewReno only, models migration as an instant switch plus one validation round trip, uses drop-tail queues and ±10% path variation, and has 20 seeds per scenario, which was too few to resolve small effects such as the loss reduction when new flows joined ($p = 0.06$). Real stacks add delayed ACKs, CUBIC or BBR, link-layer handover gaps and NAT rebinding, any of which could change the size of MACR's benefit. The Wireshark experiment, in turn, captured only connection set-up on a short-RTT path; it does not show the slow-start ramp after a migration, which the model covers instead.

VI. CONCLUSION AND FUTURE SCOPE

The reviewed literature agrees that QUIC improves on TCP+TLS where its design differs structurally: one-round-trip set-up, per-stream delivery and survival of address changes. It

disagrees on how much of QUIC's measured performance is due to the protocol rather than to the congestion controller and implementation it ships with, and we conclude that fair comparisons must hold the controller fixed. The clearest open gap concerns mobility: QUIC keeps a connection alive across migration but forgets everything it learned about each path. Our model indicates that reusing that knowledge through Careful Resume's safety phases removes overshoot losses and shortens ramp-up on high-BDP paths, at the cost of slower convergence against established competitors.

Future work should (i) implement MACR in an open-source QUIC stack such as picoquic, which already contains a Careful Resume implementation, (ii) evaluate it with CUBIC and BBR in an emulator such as Mininet and on real Wi-Fi/cellular handovers, (iii) study clients behind carrier-grade NAT, whose returning address may not match the cached entry, and (iv) compare it with the multipath QUIC extension once that is standardised [16].

References

[1] A. Langley et al., "The QUIC transport protocol: Design and Internet-scale deployment," in Proc. ACM SIGCOMM, 2017, pp. 183–196, doi: 10.1145/3098822.3098842.

[2] J. Iyengar and M. Thomson, Eds., "QUIC: A UDP-based multiplexed and secure transport," RFC 9000, IETF, May 2021, doi: 10.17487/RFC9000.

[3] M. Bishop, Ed., "HTTP/3," RFC 9114, IETF, Jun. 2022, doi: 10.17487/RFC9114.

[4] E. Rescorla, "The Transport Layer Security (TLS) protocol version 1.3," RFC 8446, IETF, Aug. 2018, doi: 10.17487/RFC8446.

[5] M. Thomson and S. Turner, Eds., "Using TLS to secure QUIC," RFC 9001, IETF, May 2021, doi: 10.17487/RFC9001.

[6] A. Yu and T. A. Benson, "Dissecting performance of production QUIC," in Proc. Web Conf. (WWW), 2021, pp. 1157–1168, doi: 10.1145/3442381.3450103.

[7] A. M. Kakhki, S. Jero, D. Choffnes, C. Nita-Rotaru, and A. Mislove, "Taking a long look at QUIC: An approach for rigorous evaluation of rapidly evolving transport protocols," in Proc. ACM IMC, 2017, pp. 290–303, doi: 10.1145/3131365.3131368.

[8] L. Xu, S. Ha, I. Rhee, V. Goel, and L. Eggert, Ed., "CUBIC for fast and long-distance networks," RFC 9438, IETF, Aug. 2023, doi: 10.17487/RFC9438.

[9] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "BBR: Congestion-based congestion control," ACM Queue, vol. 14, no. 5, pp. 20–53, 2016, doi: 10.1145/3012426.3022184.

[10] J. Iyengar and I. Swett, Eds., "QUIC loss detection and congestion control," RFC 9002, IETF, May 2021, doi: 10.17487/RFC9002.

[11] M. Mazilu, A. Valentine, I. Wakeman, and G. Parisis, "An experimental study of congestion control in LEO satellite networks," in Proc. IFIP Networking Conf., 2026.

[12] A. Ford, C. Raiciu, M. Handley, O. Bonaventure, and C. Paasch, "TCP extensions for multipath operation with multiple addresses," RFC 8684, IETF, Mar. 2020, doi: 10.17487/RFC8684.

[13] A. Buchet and C. Pelsser, "An analysis of QUIC connection migration in the wild," ACM SIGCOMM Comput. Commun. Rev., vol. 55, no. 1, pp. 3–9, 2025, doi: 10.1145/3727063.3727066.

[14] L. Tan, X. Dong, X. Gao, P. Zhang, and W. Su, "Enhancing QUIC performance in heterogeneous networks: A proactive connection migration approach," Int. J. Netw. Manage., vol. 35, no. 5, e70022, 2025.

[15] S. Y. Kim and S. J. Koh, "Adaptive control of congestion window in QUIC," in Proc. 14th Int. Conf. Inf. Commun. Technol. Convergence (ICTC), 2023, pp. 1394–1396, doi: 10.1109/ICTC58733.2023.10393389.

[16] Y. Liu, Y. Ma, Q. De Coninck, O. Bonaventure, C. Huitema, and M. Kühlewind, "Managing multiple paths for a QUIC connection," Internet-Draft draft-ietf-quic-multipath-21, IETF, Mar. 2026, work in progress.

[17] X. Cao and X. Zhang, "SaTCP: Link-layer informed TCP adaptation for highly dynamic LEO satellite networks," in Proc. IEEE INFOCOM, 2023.

[18] V. Kamel, J. Zhao, D. Li, and J. Pan, "StarQUIC: Tuning congestion control algorithms for QUIC over LEO satellite networks," in Proc. ACM LEO-NET, 2024.

[19] Z. Lai et al., "LeoCC: Making Internet congestion control robust to LEO satellite dynamics," in Proc. ACM SIGCOMM, 2025, doi: 10.1145/3718958.3750491.

[20] picoquic project, "Migration to a new IP address keeps the old path's congestion window and RTT (RFC 9000 Sec. 9.4)," GitHub issue #2237, 2026. [Online]. Available: https://github.com/private-octopus/picoquic/issues/2237

[21] J. Chu, N. Dukkipati, Y. Cheng, and M. Mathis, "Increasing TCP's initial window," RFC 6928, IETF, Apr. 2013, doi: 10.17487/RFC6928.

[22] P. Balasubramanian, Y. Huang, and M. Olson, "HyStart++: Modified slow start for TCP," RFC 9406, IETF, May 2023, doi: 10.17487/RFC9406.

[23] J. Touch, M. Welzl, and S. Islam, "TCP control block interdependence," RFC 9040, IETF, Jul. 2021, doi: 10.17487/RFC9040.

[24] G. Fairhurst, A. Sathiaseelan, and R. Secchi, "Updating TCP to support rate-limited traffic," RFC 7661, IETF, Oct. 2015, doi: 10.17487/RFC7661.

[25] N. Kuhn, E. Stephan, G. Fairhurst, R. Secchi, and C. Huitema, "Careful Resume: Convergence of congestion control from retained state," RFC 9959, IETF, May 2026, doi: 10.17487/RFC9959.

[26] M. Yanev, A. Custura, R. Secchi, and G. Fairhurst, "Analysis of careful resumption of Internet congestion control from retained path state," Computer Networks, vol. 276, 2026, doi: 10.1016/j.comnet.2025.111950.